Community Detection in Bacterial and Viral Networks

Jonas Ziegler

March 1, 2026

Abstract

This paper investigates the community structure of bacterial and viral protein similarity networks using modularity-based community detection methods. We compare the Louvain and Leiden algorithms and assess the similarity of their resulting partitions using the Adjusted Rand Index (ARI).

1 Introduction

1.1 Motivation

1.1.1 Why statistical network analysis is important

We live in a highly interconnected world where many physical, social, technological and biological systems consist of agents or entities interacting with each other. Examples include a virus being transmitted over social contact networks, global trade between countries, and the human brain. Any such system can be represented as a network by denoting the entities as nodes and the interactions between them as edges. This makes networks an important type of data spanning a remarkable variety of complex systems. It is therefore very important to have mathematically rigorous and practically useful methods for statistical analysis of networks.

1.1.2 Biological Networks

Biological systems are often represented as networks. Essentially, every biological entity has interactions with other biological entities, from the molecular to the ecosystem level, providing us with the opportunity to model biological systems using many different types of networks such as ecological, neurological, metabolic, and protein–protein interaction networks.

A protein-protein interaction network (PPI network), also known as the interactome, is a type of biological network where a node represents a protein and an edge represents the physical interaction between each pair of proteins.

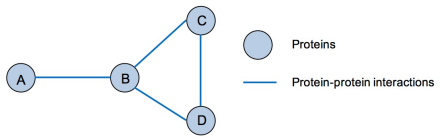


Figure 1: Protein–protein interaction network. Adapted from European Bioinformatics Institute (EMBL-EBI) 2024.

1.1.3 Why Community Detection

M. E. J. Newman (2006) argues that detecting and characterizing community structure is one of the fundamental problems in the study of networked systems, and notes that many networks of scientific interest — including biological ones — naturally divide into communities.

naturally divide into communities. Detecting these communities can help us understand

the structure and function of the network, as well as the underlying system it represents. For example, in the context of PPI (protein–protein interaction) networks, communities can help identify unknown protein complexes.

1.2 Definition of a Network

1.2.1 Graph as a mathematical object

We call a collection of connected objects a network. In the field of mathematics, a network can also be called a graph. The mathematical theory behind networks is known as graph theory. A graph is a mathematical object that consists of a set of objects (called vertices or nodes) that are connected by links (called edges).

1.2.2 Adjacency list

An adjacency list is a graph representation method that uses an array of lists to store connections between vertices. Each array index represents a vertex, and its corresponding list contains all neighboring nodes, making it highly efficient for sparse graphs($G = (V, E)$, where the number of edges is much less than the maximum possible number of edges). The main operation performed by the adjacency list data structure is to report a list of neighbors of a given vertex.

1.2.3 Adjacency matrix

The main alternative to the adjacency list is the adjacency matrix, a matrix whose rows and columns are indexed by vertices and the cell (i, j) contains 1 if there is an edge between vertices i and j, or 0 if no such edge exists. The diagonal elements of the matrix are all 0, since edges from a vertex to itself are not allowed in simple graphs.

1.2.4 Directed vs. undirected networks

Undirected and directed graphs are fundamental concepts in graph theory. An undirected graph is a graph where the edges do not have a specific direction and it is bidirectional in nature meaning it does not have a parent-child relation concept. In contrast, a directed graph is a graph where the edges have a specific direction, indicating a one-way relationship between the vertices.

An important difference between directed and undirected graphs is that in a undirected graph, relationships are symmetric, meaning if vertex A is connected to vertex B, then vertex B is also connected to vertex A. In a directed graph, relationships are asymmetric, meaning if vertex A is connected to vertex B, it does not necessarily imply that vertex B is connected to vertex A. This characteristic is well represented in the adjacency matrix as you can see in the figure below.

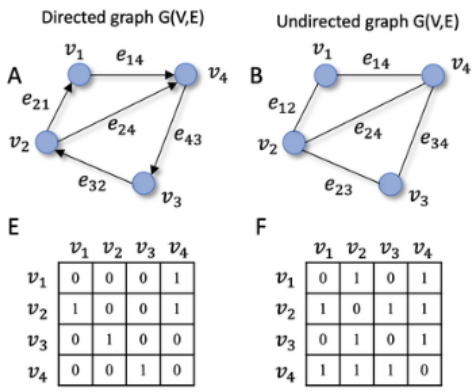


Figure 2: Directed vs.undirected graph. Adapted fromResearchGate 2020.

1.2.5 Weighted undirected network

Because in this paper we will be working with weighted undirected networks, we will now define what a weighted undirected network is. A weighted undirected network is a graph where each edge has a weight associated with it, representing the strength or importance of the connection between two vertices. In such networks, the adjacency matrix is not binary but contains numerical weights for each edge. Mathematically, a weighted undirected network can be represented as $G = (V, E, W)$, where V is the set of vertices, E is the set of edges, and W is a function that assigns a weight to each edge in E.

1.3 Network Analysis

1.3.1 Descriptive statistics

Hevey (2018) states that Network analysis provides the capacity to estimate complex patterns of relationships, and the network structure can be analyzed to reveal core features of the network. A variety of basic descriptive metrics can be used to characterize network topology, including number of nodes and edges, network density, degree distribution and centrality measures. These measures provide an initial understanding of connectivity patterns and structural organization.

1.3.2 Community Detection and Community Comparison

In network science, a community is a group of nodes that are more densely connected to each other than to the rest of the network. Detecting communities can help us understand the functional organization of a system. For example, in a PPI network, groups of proteins that interact often perform the same function in a cell. Community detection methods such as Edge-Betweenness, Louvain and Leiden algorithms can be used to identify these community structures. For this paper, attention is given to the Adjusted Rand Index (ARI) as a similarity measure for community comparison, which quantifies the agreement between different community structures. This concept will be explained in more detail in the following sections.

2 Network Summary Statistics

2.1 Basic Metrics

2.1.1 Number of nodes and edges

The number of nodes and edges are fundamental metrics that provide a basic understanding of the size and complexity of a network. The number of nodes, denoted as $|V|$, represents the total

count of distinct entities in the network, while the number of edges, denoted as $|E|$, represents the total count of connections between those entities.

2.1.2 Density

Network density is a measure of how many edges are in the network compared to the maximum possible number of edges. The metric is defined as the ratio of observed edges to the total possible connections, ranging from 0 to 1, where 0 indicates a completely disconnected network and 1 indicates a fully connected network. For an undirected network, the density can be calculated using the formula:

$$\text{Density} = \frac{|E|}{\binom{|V|}{2}} = \frac{2|E|}{|V|(|V|-1)}$$

2.1.3 Average degree

The average degree of a network is a network metric representing the mean number of connections per node. The degree of a node refers to the number of edges connected to that node. These edges are called incident edges, meaning edges that are directly connected to the node in question. In an undirected network, the degree simply counts all incident edges, while in a directed network, a distinction is made between in-degree and out-degree. For an undirected network, it can be expressed mathematically as:

$$\text{Average Degree} = \frac{2|E|}{|V|}$$

For a directed network, it can be expressed mathematically as:

$$\text{Average In-Degree} = \text{Average Out-Degree} = \frac{|E|}{|V|}$$

Average In-Degree means the average number of incoming edges per node, while Average Out-Degree means the average number of outgoing edges per node. They are equal in a directed network because each edge contributes to both an in-degree and an out-degree.

2.2 Degree Distribution

2.2.1 Definition of degree distribution

The degree distribution of a network describes the probability distribution of the degrees of nodes in a network. It specifies the probability P(k) that a randomly selected node has exactly k connections (or degree k). The distribution provides insight into the structural organization of a network and allows the differentiation between homogeneous and heterogeneous networks.

2.2.2 Random graph and scale-free graph

Random graphs and scale-free graphs are two fundamental network models that differ significantly in their degree distributions. A random graph, often modeled using the Erdős–Rényi framework, is constructed by connecting nodes randomly with a fixed probability p. In such networks, edges are formed independently of each other. As a result, most nodes tend to have a similar number of connections, leading to a relatively homogeneous structure. Highly connected nodes (hubs) are extremely rare in random graphs. The degree distribution of a random graph follows a Poisson distribution, which implies that most nodes have a degree close to the average degree. The probability function for a Poisson distribution is given by:

$$P(k) = \frac{\lambda^k e^{-\lambda}}{k!}$$

where λ is the average degree of the network.

Scale-free graphs, on the other hand, exhibit a highly heterogeneous structure. These networks grow over time, and new nodes preferentially attach to already well-connected nodes. This process leads to the emergence of hubs, which are nodes with a significantly higher degree than the average node. Consequently, scale-free networks are characterized by strong degree inequality. Scale-free networks follow a power-law degree distribution of the form:

$$P(k) \sim k^{-\gamma}$$

where γ is a positive constant typically in the range of 2 to 3. This power-law behavior indicates that while most nodes have a low degree, a few nodes (hubs) have a very high degree. Below is a visual representation of the degree distribution for random and scale-free networks.

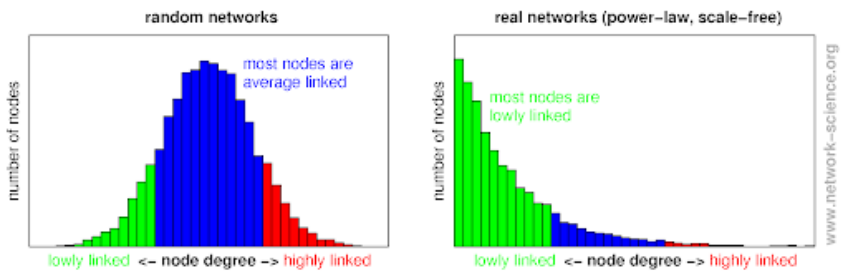


Figure 3: Comparison of random and scale-free networks and their corresponding degree distributions. Adapted from Science n.d.

2.2.3 Structural Implications

The degree distribution of a network has significant implications for its structure and dynamics. In networks with a Poisson degree distribution, connectivity is relatively uniform, meaning most nodes have the same number of connections and there are rarely outliers. Therefore, the network does not depend heavily on individual nodes. In contrast, scale-free networks are dominated by hubs, which play a crucial role in maintaining overall connectivity. The structural difference affects various network properties. For instance, scale-free networks are typically robust against random node failures, as most nodes are low-degree nodes. However, they are highly vulnerable to targeted attacks on hubs. Furthermore, the presence of hubs strongly influences centrality measures and also modularity. As a result, understanding the degree distribution is essential for interpreting community detection results and structural properties. Centrality measures and Modularity will be discussed in the following sections.

2.3 Centrality Measures

2.3.1 Degree centrality

Degree centrality measures a node's importance in a network by counting its total number of connections (degree). In an undirected network, the degree centrality of a node is simply the number of edges connected to it. It can be expressed mathematically as:

$$C_D(v) = \deg(v)$$

where $\deg(v)$ is the degree of node v. In a directed network, degree centrality can be further divided into in-degree centrality and out-degree centrality. It can be expressed mathematically as:

$$C_{D_{in}}(v) = \deg_{in}(v)$$

$$C_{D_{out}}(v) = \deg_{out}(v)$$

where $\deg_{in}(v)$ is the number of incoming edges to node v and $\deg_{out}(v)$ is the number of outgoing edges from node v.

2.3.2 Betweenness centrality

Betweenness centrality measures the importance of a node by quantifying how often it lies on shortest paths between other pairs of nodes. More formally, it counts the proportion of shortest paths between all node pairs that pass through a given node. Mathematically, the betweenness centrality of a node v is defined as:

$$C_B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}}$$

where σ_{st} is the total number of shortest paths from node s to node t, and $\sigma_{st}(v)$ is the number of those paths that pass through node v. Nodes with a high betweenness centrality often act as bridges between different parts of the network, frequently connecting otherwise weakly connected communities. Edge betweenness extends the same idea to edges instead of nodes. It measures how often an edge lies on the shortest paths between pairs of nodes. The betweenness centrality of an edge e is defined as:

$$C_B(e) = \sum_{s \neq t} \frac{\sigma_{st}(e)}{\sigma_{st}}$$

where σ_{st} is the total number of shortest paths from node s to node t, and $\sigma_{st}(e)$ is the number of those paths that pass through edge e. This characteristic is particularly important in community detection methods based on edge betweenness, such as the algorithm introduced by M. G. Newman M. E. J. (2002), which will be discussed in more detail in the section on community detection methods.

2.4 Modularity

The core idea behind modularity is that a network has a strong community structure if:

- there are many edges between nodes within the same community.
- there are few edges between nodes in different communities.

Therefore, modularity compares the observed structure to a random baseline. This means that modularity measures how much more densely connected nodes within the same community are than we would expect by chance. The modularity formula is given by:

$$Q = \frac{1}{2m} \sum_{ij} (A_{ij} - P_{ij}) \delta(c_i, c_j)$$

where m is the total number of edges in the network, A_{ij} is the adjacency matrix element representing the presence of an edge between nodes i and j, P_{ij} is the expected number of edges between nodes i and j in a random graph with the same degree distribution, and $\delta(c_i, c_j)$ is the Kronecker delta function that equals 1 if nodes i and j are in the same community (i.e., $c_i = c_j$) and 0 otherwise. The modularity value theoretically ranges between -1 and 1, although in practice it typically lies between 0 and 1. Higher modularity values indicate a stronger community structure, meaning that more edges fall within communities than would be expected in a comparable random network. A known limitation of modularity is the resolution limit problem, meaning that small communities may not be detected because they do not contribute enough to the overall modularity score.

2.5 Clustering Coefficient

2.5.1 Definition and interpretation

A third property that many networks have in common is clustering, or network transitivity, which is the property that two vertices that are both neighbors of the same third vertex have a heightened probability of also being neighbors of one another. For example, in the context of a protein–protein interaction network, if two proteins interact with the same third protein, they are more likely to interact with each other as well. This effect is quantified by the clustering coefficient C, defined by:

$$C = \frac{3 \times \text{number of triangles}}{\text{number of connected triples}}$$

where a triangle is a set of three nodes that are mutually connected, and a connected triple is a set of three nodes where at least two of them are connected.

3 Similarity Measures for Community Comparison

3.1 Adjusted Rand Index (ARI)

The Adjusted Rand Index (ARI) is a metric that measures the similarity between two partitions of the same set of nodes. A partition refers to the complete division of nodes into clusters. The ARI considers all pairs of nodes and evaluates whether each pair is assigned consistently to the same community or to different communities in both partitions.

The classical Rand Index (RI) evaluates the proportion of node pairs that are either placed in the same cluster in both partitions or in different clusters in both partitions. However, since random clusterings can also exhibit non-zero agreement, the ARI corrects this value by adjusting for chance.

Formally, the ARI is defined as:

$$ARI = \frac{RI - E[RI]}{\max(RI) - E[RI]} \tag{1}$$

where RI is the Rand Index, $E[RI]$ is the expected value of the Rand Index for random clusterings, and $\max(RI)$ is the maximum possible Rand Index. The ARI ranges from -1 to 1, where:

- 1 indicates perfect agreement between the two partitions,
- 0 indicates that the agreement is no better than random,
- Negative values indicate less agreement than expected by chance.

3.2 Adjusted Mutual Information (AMI)

The Adjusted Mutual Information (AMI) is another metric used to compare two partitions based on information theory. It measures the mutual information between two clusterings, quantifying how much knowing one partition helps to predict the cluster assignments of the other.

Mutual Information (MI) measures the amount of information shared between two partitions. However, like the RI, MI can be biased by random agreement. The AMI corrects for this by subtracting the expected mutual information under a random model.

The AMI is defined as:

$$AMI = \frac{MI - E[MI]}{\max(MI) - E[MI]} \tag{2}$$

where MI is the Mutual Information, $E[MI]$ is the expected value of the Mutual Information for random clusterings, and $\max(MI)$ is the maximum possible Mutual Information. The AMI also ranges from -1 to 1, with similar interpretations as the ARI:

- 1 indicates perfect agreement,
- 0 indicates agreement no better than random,
- Negative values indicate less agreement than expected by chance.

3.3 Comparison of ARI and AMI

Both ARI and AMI are widely used for evaluating the similarity between two partitions; however, they differ conceptually and computationally. The ARI is pair-counting-based and focused on whether pairs of nodes are assigned consistently across partitions. It is relatively easy to interpret. However, it can be sensitive to imbalanced cluster sizes. On the other hand, AMI is information-theoretic and evaluates how much information is shared between clusterings. It is generally more robust to imbalanced cluster distributions and performs well when the number of clusters differs between the partitions. The ARI is often preferred when interpretability and pairwise consistency are important, particularly in the case of balanced cluster sizes.

The AMI is advantageous when cluster sizes vary significantly or when comparing partitions with different numbers of communities. Due to its information-theoretic nature, it provides a more distribution-sensitive comparison. Both indices are valuable tools for community comparison. Later in the results section, we will apply ARI to compare different community detection results.

4 Community Detection Methods

In this section, we will explore the community detection methods that form the core of this study.

4.1 Edge Betweenness

The edge betweenness method, introduced by Girvan and Newman (2002), was developed to overcome the shortcomings of traditional community detection methods such as the hierarchical clustering approach. The hierarchical clustering method focuses on constructing a measure that indicates which edges are most central to communities. Using these measures, it constructs communities by adding the strongest edges to an initially empty graph. However, this method

has several drawbacks, such as relying on a subjective choice of similarity measure. Furthermore, because it tries to build communities by adding edges, early mistakes caused by noise in the data cannot be corrected later on.

The edge betweenness method, on the other hand, removes weak edges from the original graph instead of adding edges. To identify weak edges in a graph, Girvan and Newman generalized Freeman's betweenness centrality to edges.

M. G. Newman M. E. J. (2002) state that if a network contains communities or groups that are only loosely connected by a few intergroup edges, then all shortest paths between different communities must pass through one of these few edges. Thus, the edges connecting communities will have high edge betweenness. By removing these edges, we separate groups from one another and thereby reveal the underlying community structure of the graph.

The algorithm they propose can be summarized as follows:

1. Compute the edge betweenness for all edges in the network.
2. Remove the edge with the highest edge betweenness.
3. Recalculate the edge betweenness for all edges affected by the removal.
4. Repeat from step 2 until no edges remain.

The iteration ends when no edges remain; therefore, in practice, we must decide when to stop the algorithm. This can be done by evaluating the modularity of the network after each iteration and stopping when the modularity is maximized.

This algorithm is computationally expensive because we must recalculate edge betweenness after each iteration, making the algorithm run in worst-case time $O(m^2n)$, where m is the number of edges and n is the number of nodes in the graph.

It may be tempting to calculate the betweenness of all edges only once and then remove them in order of decreasing betweenness. However, this approach does not work well, because if two communities are connected by more than one edge, there is no guarantee that all of these edges will have high betweenness. By recalculating betweenness after the removal of each edge, we can ensure that at least one of the remaining edges between two communities will always have high betweenness.

4.2 Louvain Method

The Louvain algorithm, introduced by Blondel et al. (2008), is a popular method for community detection in large networks. The core idea is to group nodes so that connections inside communities are denser than expected at random. Louvain achieves this through greedy local optimization of the modularity measure, combined with a hierarchical approach.

The algorithm consists of two phases that are repeated iteratively until no further improvement in modularity is possible:

1. **Modularity Optimization:**
   - Initially, each node is assigned to its own community.
   - For each node i, we consider moving it to the community of each neighbor.
   - The change in modularity ΔQ is calculated for each possible move, and the node is moved to the community that results in the highest increase in modularity. ΔQ is given by the formula:

$$\Delta Q = Q(i \in C) - Q(i \in A)$$

where $Q(i \in C)$ is the modularity of the network if node i is moved to community C, and $Q(i \in A)$ is the modularity if node i remains in its current community A. The formula for calculating modularity is given by:

$$Q = \frac{1}{2m} \sum_{ij} \left[A_{ij} - \gamma \frac{k_i k_j}{2m} \right] \delta(c_i, c_j)$$

where A_{ij} is the adjacency matrix of the graph, k_i and k_j are the degrees of nodes i and j, m is the total number of edges in the graph, c_i and c_j are the communities of nodes i and j, and $\delta(c_i, c_j)$ is the Kronecker delta function, which equals 1 if $c_i = c_j$ and 0 otherwise. γ is the resolution parameter that allows control over the size of the detected communities. If $\gamma = 1$, we obtain the standard modularity. If $\gamma < 1$, larger communities are detected, and if $\gamma > 1$, smaller communities are detected.

2. **Community Aggregation:**

   - After no further individual node movements can increase modularity, we create a new network in which each community is treated as a single node, also known as a supernode.
   - Edges between supernodes are replaced by weighted edges, where the weight of the edge between two supernodes is the sum of the weights of the edges between the nodes in the corresponding communities.
   - Edges within the same community are represented as self-loops.

In the figure below, we can see a visualization of the Louvain method.

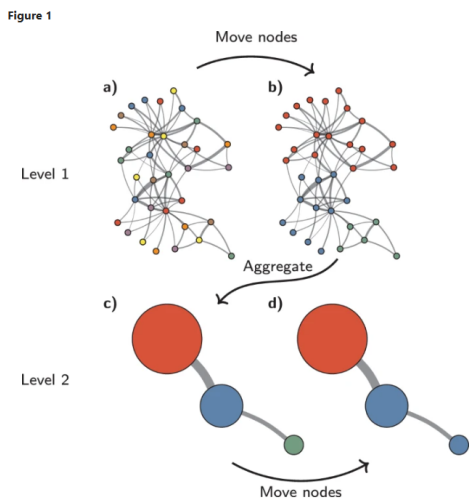


Figure 4: Visual representation of the Louvain method for community detection. Adapted from V.A. Traag, L. Waltman, N.J. van Eck 2019.

The hierarchical nature of the Louvain method allows it to capture community structure at multiple scales, and it is computationally efficient, with a time complexity of $O(n \log n)$ in practice, making it suitable for large networks.

However, there are several notable limitations to the Louvain method. First, it does not guarantee that the detected communities are internally connected. In practice, a community may consist of multiple disconnected components. Furthermore, the algorithm may converge to a local maximum of modularity, as it only ensures that no single node move can further increase modularity. These shortcomings motivated Traag, Waltman, and van Eck (2019) to introduce the Leiden algorithm, which is discussed in the following section.

4.3 Leiden Method

We now introduce the Leiden algorithm, which guarantees well-connected communities. The algorithm is partly based on the previously introduced smart local moving algorithm, which is a faster version of the Louvain algorithm. The Leiden algorithm also takes advantage of the idea of speeding up the local moving of nodes and the idea of moving nodes to a random neighbor by only considering a subset of nodes that are likely to move. V.A. Traag (2019a) state that they consider these ideas to represent the most promising directions in which the Louvain algorithm can be improved, even though they recognize that other improvements have been suggested as well.

The Leiden algorithm consists of three phases:

1. **Local Moving of Nodes:**
   - Similar to the Louvain method, each node is initially assigned to its own community.
   - For each node i, we consider moving it to the community of each neighbor and calculate the change in modularity ΔQ for each possible move.
   - The node is moved to the community that results in the highest increase in modularity, but only if this move does not lead to disconnected communities.
2. **Refinement of Communities:**
   - After no more individual node movements can increase modularity, we refine the communities by splitting them into smaller subcommunities if this leads to an increase in modularity.
3. **Community Aggregation:**
   - Similar to the Louvain method, we create a new network in which each community is treated as a single node, and edges between supernodes are replaced by weighted edges.

In the figure below, we can see a visualization of the Leiden method.

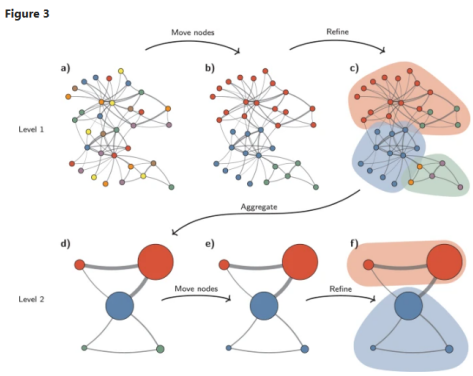


Figure 5: Visual representation of the Leiden method for community detection. Adapted from V.A. Traag 2019b.

V.A. Traag (2019a) explain that, in contrast to the Louvain algorithm, the Leiden algorithm introduces an additional refinement phase between the local moving phase and the aggregation phase. The purpose of this refinement step is to construct a partition that refines the partition obtained after the local moving phase. More precisely, if the partition resulting from the local moving phase is denoted by P, the refinement phase identifies a new partition P', where each

community in P' is a subset of a community in P. Thus, communities in P may be subdivided into smaller, more internally cohesive subcommunities in P'.

Initially, the refined partition P' assigns each node to its own singleton community. Subsequently, nodes are merged within their original community in P, but only if such merges improve the quality function. Importantly, merging is restricted to nodes that belong to the same community in P, ensuring that P' remains a refinement of P. After the refinement phase, communities in P' may consist of multiple nodes, yet they remain strictly contained within the original communities of P.

Compared to the Louvain algorithm, this refinement phase provides greater flexibility in exploring alternative partitions. Whereas Louvain performs greedy node movements without revisiting earlier decisions, Leiden systematically evaluates potential merges within communities. Merges are only accepted if they increase the quality function, thereby preventing deteriorations in partition quality. As a result, the algorithm is able to escape poorly connected or suboptimal community configurations.

Another important difference concerns the local moving phase itself. In the Louvain algorithm, nodes are considered in a fixed order, and once a node is moved, it is not reconsidered unless it is affected by subsequent movements. In contrast, the Leiden algorithm maintains a queue of nodes that require reevaluation. Whenever a node changes community, its neighbors are added to the queue to ensure that their potential moves are reconsidered. This dynamic updating procedure continues until no further improvements can be achieved. By revisiting affected nodes, Leiden avoids prematurely fixing suboptimal assignments and improves convergence toward higher-quality partitions.

4.3.1 Theoretical Guarantees of the Leiden Algorithm

The Leiden algorithm is executed iteratively, where the partition obtained in one iteration serves as the starting point for the next. This iterative structure allows the algorithm to progressively refine the community structure and provides several theoretical guarantees at different stages of convergence. Below, we give an intuitive overview of these guarantees. Formal definitions and mathematical proofs can be found in the supplementary material.

After each iteration of the Leiden algorithm, the following properties hold:

1. All communities are γ-separated.
2. All communities are γ-connected.

Here, γ denotes the resolution parameter of the quality function being optimized, such as modularity. The property of γ-separation implies that no pair of communities can be merged without decreasing the quality function. This property is also satisfied by the Louvain algorithm.

The property of γ-connectivity is stronger than ordinary connectivity. It guarantees that communities are internally well connected in a manner consistent with the resolution parameter. In contrast, the Louvain algorithm does not guarantee connectivity and therefore does not ensure γ-connectivity.

An iteration in which the partition remains unchanged is referred to as a *stable iteration*. After a stable iteration of the Leiden algorithm, two additional guarantees hold:

1. All nodes are locally optimally assigned.
2. All communities are subpartition γ-dense.

Local optimality of nodes means that no individual node can be moved to another community in a way that improves the quality function. This property is also guaranteed after a stable iteration of the Louvain algorithm.

Subpartition γ-density, however, is not guaranteed by Louvain. A community is subpartition γ-dense if it cannot be decomposed into internally separable parts without reducing the quality function. In other words, each community consists of well-connected components that themselves satisfy the same density condition recursively. This property ensures structural cohesion beyond simple node-level optimality.

Unlike the Louvain algorithm, the Leiden algorithm may still improve the partition even after a stable iteration. When iterated until full convergence, the Leiden algorithm yields a partition satisfying two stronger guarantees:

1. All communities are uniformly γ-dense.
2. All communities are subset optimal.

Uniform γ-density means that no subset of a community can be separated from it without decreasing the quality function. Regardless of how a community is divided into two parts, these parts remain sufficiently well connected. This implies that the overall partition is close to optimal with respect to the quality function.

Subset optimality represents the strongest guarantee of the Leiden algorithm. It states that every subset of every community is locally optimally assigned; that is, no subset of nodes can be moved to another community in a way that improves the objective function. Subset optimality implies uniform γ-density and all previously mentioned properties.

In summary, while the Louvain algorithm guarantees only local node-level optimality and γ-separation, the Leiden algorithm provides substantially stronger guarantees, ensuring well-connected, structurally cohesive, and near-optimal communities.

Here is an overview of the guarantees provided by the Louvain algorithm and the Leiden algorithm.

		Louvain	Leiden
Each iteration	γ-separation	✓	✓
	γ-connectivity		✓
Stable iteration	Node optimality	✓	✓
	Subpartition γ-density		✓
Asymptotic	Uniform γ-density		✓
	Subset optimality		✓

Figure 6: Visual representation of the guarantees of the Leiden algorithm. Adapted from **OverviewGuarantees**.

Due to its stronger theoretical guarantees and improved partition quality, the Leiden algorithm is widely regarded as a state-of-the-art method for community detection in large-scale networks.

5 Data Description

5.1 Data Source

Bacterial proteins for the bac network were extracted from full bacterial sequences using Prodigal. Viral proteins for the vir network were extracted from viral metagenomic contigs after de novo assembly using Prodigal.

Protein similarity networks were constructed using MMseqs2, a sequence comparison tool designed for fast and sensitive large-scale sequence alignment. Pairwise protein similarities were computed, and an edge between two proteins was included in the network if their bit score exceeded a threshold of 50.

The resulting networks therefore represent weighted protein similarity graphs based on sequence homology. The data originally come from the paper:

Whole-Virome Analysis Sheds Light on Viral Dark Matter in Inflammatory Bowel Disease

5.2 Descriptive Comparison of Bac and Vir

The bac network consists of 819 nodes and 88,914 edges, while the vir network consists of 1,603 nodes and 25,507 edges, meaning the vir network is approximately twice as large as the bac network. However, the bac network is substantially denser.

The density of the bac network is computed as follows:

$$\text{Density} = \frac{2E}{N(N-1)} = \frac{2 \cdot 88914}{819 \cdot 818} \approx 0.265$$

meaning that 26.5% of all possible edges are present in the bac network.

In contrast, the density of the vir network is:

$$\text{Density} = \frac{2E}{N(N-1)} = \frac{2 \cdot 25507}{1603 \cdot 1602} \approx 0.0199$$

meaning that only 1.99% of all possible edges are present in the vir network.

The average degree of the bac network is:

$$\text{Average Degree} = \frac{2E}{N} = \frac{2 \cdot 88914}{819} \approx 217.1$$

while the average degree of the vir network is:

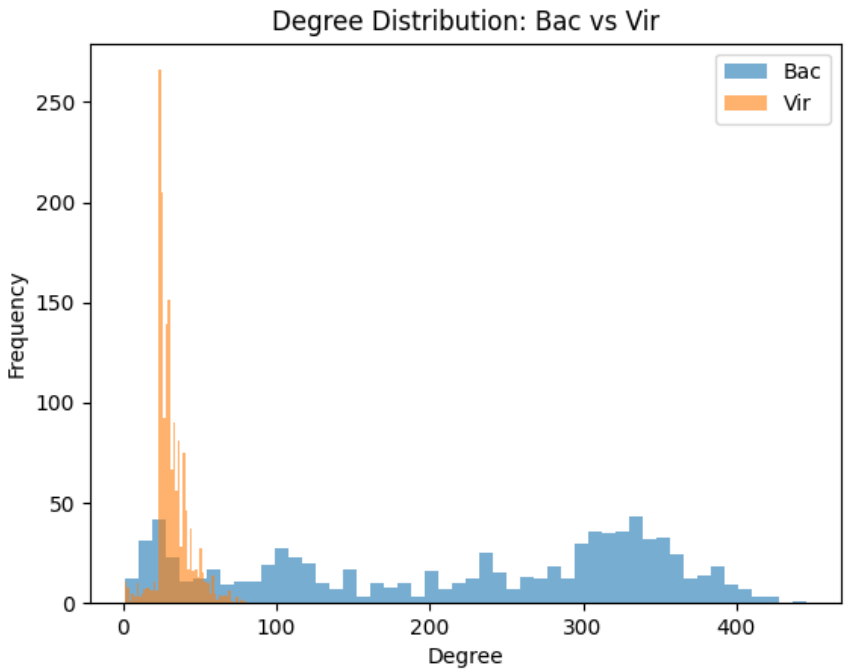
$$\text{Average Degree} = \frac{2E}{N} = \frac{2 \cdot 25507}{1603} \approx 31.8$$

meaning that, on average, bac proteins are approximately seven times more similar to other proteins than vir proteins.

Based on the substantially higher average connectivity of the bacterial network, its degree distribution is expected to be comparatively homogeneous and concentrated around higher degree values. The dense connectivity suggests the presence of strongly interconnected subclusters, potentially approaching quasi-complete structures within communities.

In contrast, the viral network is expected to exhibit a more heterogeneous and potentially heavy-tailed degree distribution. Such a structure would be characterized by a small number of highly connected hub nodes and a large proportion of weakly connected proteins, reflecting a sparse and modular topology.

By computing the degree distribution of both networks, the structural differences can be visualized using a histogram:



The histogram clearly shows that the bacterial network is characterized by substantially higher node degrees compared to the viral network. While the viral network exhibits a strong concentration of low-degree nodes, the bacterial network displays a broad distribution extending to very high degree values. This indicates a much denser and more interconnected structure in the bacterial network, whereas the viral network appears sparse and modular.

5.3 Network Construction

The networks are constructed based on sequence similarity between proteins. Nodes represent proteins, and edges represent sequence similarity between them. The weight of each edge is calculated using the bit score, which is a normalized measure of sequence similarity derived from the raw alignment score.

The bit score is computed using the following formula:

$$S' = \frac{\lambda S - \ln K}{\ln 2}$$

where S is the raw alignment score, λ and K are statistical parameters derived from the scoring system used in the sequence alignment, and ln denotes the natural logarithm.

The bit score provides a standardized way to compare sequence similarities across different alignments, allowing for a more meaningful interpretation of the edge weights in the context of protein similarity networks.

6 Experimental Setup

6.1 Implementation

After describing the data and the community detection methods, we now apply these methods to analyze the bacterial and viral protein similarity networks.

All analyses were implemented in Python. Graph construction and basic network measures were computed using the NetworkX library, while the Leiden algorithm was executed using the `leidenalg` package in combination with `igraph`.

The full source code is publicly available at: `https://github.com/j0nascz/Paper_SMNCA.git`

6.2 Evaluation Criteria

The goal of this analysis is to compare the three community detection methods (Edge Betweenness, Louvain, and Leiden) in terms of their ability to identify meaningful communities in the bac and vir networks.

However, we cannot apply the edge betweenness approach proposed by Girvan and Newman, as it is computationally infeasible for networks of this size. Consequently, we focus on comparing the Louvain and Leiden methods.

The evaluation is based on a pairwise comparison of the resulting partitions using the Adjusted Rand Index (ARI) and the Adjusted Mutual Information (AMI).

7 Results

7.1 Method Comparison

The results of the Leiden and Louvain algorithms applied to the Bac and Vir networks are summarized in the following table:

Table 1: Community detection results for bacterial and viral networks.

Dataset	Method	# Communities	Modularity	Runtime (s)
Bac	Louvain	5	0.374	0.340
Bac	Leiden	6	0.333	0.017
Vir	Louvain	23	0.892	0.136
Vir	Leiden	23	0.884	0.007

For the Bac network, the Louvain algorithm identified 5 communities, whereas the Leiden algorithm detected 6 communities. The slight increase in the number of communities produced by Leiden suggests a refinement of the partition obtained by Louvain. This behavior is consistent with the theoretical properties of the Leiden algorithm, which aims to ensure better-connected and more internally coherent communities.

In terms of modularity, Louvain achieved a value of approximately 0.374, while Leiden reached approximately 0.333. The higher modularity value for Louvain in the bacterial network suggests that Louvain is more effective at optimizing the modularity objective in this specific case. This indicates that, although Leiden produces a slightly more refined partition, the Louvain partition achieves stronger optimization of the modularity objective under the given settings.

For the Vir network, which has approximately twice as many nodes as the Bac network, both Louvain and Leiden identified 23 communities in the viral network. The strong agreement between the two methods indicates that the underlying community structure is clearly defined and does not depend strongly on the specific optimization strategy.

In terms of modularity, Louvain achieved a value of approximately 0.892, while Leiden obtained approximately 0.884. Although Louvain reaches a slightly higher modularity score, the difference between the two methods is small. This indicates that both algorithms produce partitions of very similar quality with respect to modularity optimization.

As expected, the runtime of the Louvain algorithm was higher than that of the Leiden algorithm in both networks. For the Bac network, Louvain required approximately 0.340 seconds, while Leiden required approximately 0.017 seconds. For the Vir network, Louvain required approximately 0.136 seconds, while Leiden required approximately 0.007 seconds.

This difference in runtime can be attributed to more efficient local moving, a better-structured aggregation phase, and faster convergence of the Leiden algorithm compared to Louvain, especially in larger networks.

7.2 ARI

The Adjusted Rand Index (ARI) further confirms the strong agreement between Louvain and Leiden. For the viral network, the ARI is approximately 0.979, indicating an almost identical partitioning of the network by both algorithms.

For the bacterial network, the ARI is slightly lower at approximately 0.958, yet it still reflects a very high degree of similarity. The minor difference can be attributed to the additional community detected by Leiden, which suggests a small refinement of the Louvain partition rather than a fundamentally different community structure.

Overall, the high ARI values demonstrate that both methods produce highly consistent community assignments across both datasets.

7.3 Stability of Communities

The observed differences between the Bac and Vir networks can be attributed to their fundamentally different structural organization. The bacterial network is denser and more strongly connected, indicating the presence of tightly interconnected protein communities. Such dense connectivity implies that many proteins are linked across different regions of the network, resulting in a cohesive structure with numerous inter-community connections.

In contrast, the viral network is sparser and contains a larger proportion of weakly connected nodes. Connections between modules are less frequent, leading to sharper boundaries between communities.

These structural differences are directly reflected in the modularity values. Modularity measures the extent to which connections are concentrated within communities rather than between them.

In the dense bacterial network, the abundance of cross-community edges reduces the contrast between intra- and inter-community connectivity. As a result, the modularity remains comparatively lower.

In the sparse viral network, the clear separation between communities leads to a higher modularity score, as the majority of connections are concentrated within communities rather than between them.

Thus, the difference in modularity between the two datasets does not indicate better algorithmic performance on the viral network, but rather reflects the intrinsic topological differences between a densely interconnected graph and a more clearly segmented modular structure.

8 Discussion

8.1 Differences Between Bac and Vir Networks

The observed differences in community structure between the Bac and Vir networks can primarily be attributed to their underlying structural organization. The Bac network is characterized by a higher edge density and stronger overall connectivity, suggesting the presence of tightly interconnected protein communities. In contrast, the Vir network exhibits a sparser structure with a larger proportion of weakly connected nodes.

8.2 Which Method Is More Stable

Stability in community detection refers to the robustness of identified partitions under methodological variation. The high agreement between Louvain and Leiden in both networks, as indicated by the ARI values, suggests that both methods produce stable community assignments. This reinforces confidence in the reliability of the detected communities.

8.3 Resolution Limit Problem

An important limitation of modularity-based community detection methods, such as Louvain and Leiden, is the resolution limit problem. Modularity optimization may fail to detect smaller but well-defined communities if merging them with larger groups results in a higher overall modularity score.

This limitation is particularly relevant for dense networks such as Bac, where small modules may be overlooked in favor of larger communities, leading to a less accurate representation of the true community structure.

Although the Leiden algorithm is designed to produce better-connected communities, it is not immune to the resolution limit problem. Therefore, the detected partitions should be interpreted as optimal with respect to the chosen resolution parameter rather than as uniquely defined ground-truth communities.

9 Conclusion

9.1 Summary of Findings

This study compared the performance of the Louvain and Leiden community detection algorithms on two biological networks: a bacterial protein–protein interaction network (Bac) and a viral protein–protein interaction network (Vir). The results, evaluated using the Adjusted Rand Index (ARI), show that both methods identify highly consistent community structures across both networks. This is also reflected in the similar number of communities detected and the comparable modularity scores.

A notable difference observed in this study is that the runtime of the Leiden algorithm was significantly lower than that of the Louvain algorithm. This efficiency gain is particularly evident in the larger viral network, where Leiden's runtime was approximately an order of magnitude faster than Louvain's.

Overall, the findings suggest that both Louvain and Leiden are effective methods for community detection in biological networks.

9.2 Outlook and Future Work

This study was originally intended to include an evaluation of the edge-betweenness algorithm by Girvan and Newman. However, due to runtime limitations, it was not possible to include it in the analysis. For future work, it may be worthwhile to explore ways to improve the computational efficiency of the algorithm. At present, it runs in time $O(n^3)$ on sparse graphs, which makes it infeasible for larger networks.

Future research could also further investigate intra-method stability by varying the resolution parameter in both the Louvain and Leiden algorithms. Additionally, alternative quality functions beyond modularity could be explored to address potential limitations such as the resolution limit problem. Finally, integrating biological validation of detected communities would provide further insight into the functional relevance of the identified modular structures.

References

European Bioinformatics Institute (EMBL-EBI) (2024). *Network analysis of protein interaction data – Types of biological networks.* Accessed: 27 February 2026. URL: `https://www.ebi.ac.uk/training/online/courses/network-analysis-of-protein-interaction-data-an-introduction/types-of-biological-networks/`.

Hevey, David (2018). "Network analysis: a brief overview and tutorial". In: *PMC.*

Newman, M. E. J. (2006). "Modularity and community structure in networks". In: *PNAS.*

Newman M. E. J., M. Girvan (2002). "Community structure in social and biological networks". In: *PNAS.*

ResearchGate (2020). *Different types of graphs and their corresponding adjacency matrix representations.* Accessed: 27 February 2026. URL: `https://www.researchgate.net/figure/Different-types-of-graphs-and-their-corresponding-adjacency-matrix-representations-The_fig1_347300725`.

Science, Network (n.d.). *Node Degree Distribution:Random networks versus scale-free power-law networks.* URL: `http://www.network-science.org/powerlaw_scalefree_node_degree_distribution.html`.

V.A. Traag L. Waltman, N.J. van Eck (2019a). "From Louvain to Leiden: guaranteeing well-connected communities". In: *Scientific Reports.* URL: `https://www.nature.com/articles/s41598-019-41695-z/tables/1`.

— (2019b). *Visual representation of the Leiden method for community detection.* URL: `https://www.nature.com/articles/s41598-019-41695-z`.

V.A. Traag, L. Waltman, N.J. van Eck (2019). *Visual representation of the Louvain method for community detection*. URL: https://www.nature.com/articles/s41598-019-41695-z.